

Теоретический материал

Алгоритмы поиска наибольшего общего делителя

НОД – наибольший общий делитель, – число, делящее без остатка пару чисел и являющееся наибольшим из возможных делителей. Например: $\text{НОД}(8, 6)=2$; $\text{НОД}(12, 8)=4$. Очень важное понятие, необходимость в вычислении которого возникает достаточно часто. Простой расчетной формулы для НОД не существует. Решение полным перебором возможно, но для больших чисел необходимо большое количество вычислительных операций.

Хорошее решение проблемы поиска наибольшего общего делителя двух чисел нашел еще Евклид. В самом простом случае алгоритм Евклида применяется к паре положительных целых чисел и формирует новую пару, которая состоит из меньшего числа и разницы между большим и меньшим числом. Процесс повторяется, пока числа не станут равными. Найденное число и есть наибольший общий делитель исходной пары. Блок-схема алгоритма выглядит следующим образом:



Для уменьшения количества вычислительных операций новая пара положительных чисел формируется не в результате вычитания из большего числа меньшего, а в результате деления большего числа на меньшее. В результате новая пара положительных чисел формируется из делителя и остатка от деления. Как только остаток от деления становится равным нулю, вычисления прекращаются.

Искомый наибольший общий делитель – это делитель на последнем шаге вычислений. Примеры для обоих подходов (вычитание и деление):

Найти наибольший общий делитель для чисел 128 и 96.

$$128 - 96 = 32$$

$$96 - 32 = 64$$

$$64 - 32 = 32$$

$$32 - 32 = 0$$

$$128 / 96 = 1 \text{ (остаток 32)}$$

$$96 / 32 = 3$$

Ответ: 32

Пример 1 демонстрирует поиск наибольшего общего делителя двух чисел методом вычитания

Алгоритмы факторизации

Факторизация – это задача разбиения числа на множители. Известно, что любое число можно представить в следующем виде $A = a_1^{k_1} a_2^{k_2} \dots a_n^{k_n}$, где a_i – простые числа, k_i – натуральные числа. Такое разложение существует всегда и называется оно **каноническим разложением числа**. Для построения канонического разложения достаточно научиться находить только один делитель. Получив делитель и поделив на него число A (до тех пор, пока это возможно, поскольку число может поделиться на найденный простой делитель несколько раз), мы переходим к задаче поиска делителя для меньшего числа. Например, если найден первый простой делитель числа A , то $A = B \cdot a_1^{k_1}$, где a_1 – простое число, а k_1 – количество раз, на которое число A поделилось. Например, число 60 делится на 2 (первое простое число) 2 раза, то есть $60 = 15 \cdot 2^2$. Далее необходимо осуществлять поиск простого делителя числа 15. Итоговое каноническое разложение числа 60 имеет вид: $60 = 2^2 \cdot 3 \cdot 5$. Таким образом, для поиска канонического разложения числа необходимо осуществить поиск именно простых делителей. Для этого можно изначально рассматривать в качестве кандидатов на делитель только простые числа, которые могут быть первоначально найдены с помощью одного из алгоритмов: например, решето Эратосфена или решето Сундарамы.

Поиск возможного делителя, если речь не идет о скорости – задача простая. Достаточно перебрать все простые числа от 2 до $\sqrt{A}$ и для каждого из них проверить делимость числа A .

Пример 2 демонстрирует алгоритм нахождения наименьшего положительного целого делителя (большего единицы) для заданного числа, а также определение того, сколько раз число будет делиться на этот делитель без остатка

Алгоритм Ферма для разделения числа на 2 множителя

Пусть $n = p \cdot q$ – известное целое число, являющееся произведением двух неизвестных простых чисел p и q , которые требуется найти. Большинство современных методов факторизации основано на идее, предложенной еще Пьером Ферма, заключающейся в поиске пар натуральных чисел A и B таких, что выполняется соотношение:

$$n = A^2 - B^2. \quad (2.20)$$

Алгоритм Ферма может быть описан следующим образом:

1. Вычислим целую часть от квадратного корня из n :

$$m = \lceil \sqrt{n} \rceil.$$

2. Для $x = 1, 2, \dots$ будем вычислять значения

$$q(x) = (m + x)^2 - n, \quad (2.21)$$

до тех пор, пока очередное значение $q(x)$ не окажется равным полному квадрату.

3. Пусть $q(x)$ является полным квадратом, например, числа B : $q(x) = B^2$. Определим $A = m + x$, откуда из равенства $A^2 - n = B^2$ найдем $n = A^2 - B^2 = (A + B) \cdot (A - B)$, и искомые делители p и q вычисляются, как $p = A + B$, $q = A - B$.

Пример. Пусть факторизируемое число $n = 19\,691$. Вычислим $m = \lfloor \sqrt{n} \rfloor = 140$. Представим процедуру вычисления делителей n в виде таблицы:

x	q	$\sqrt{q}$
1	190	13,78
2	473	21,75
3	758	27,53
4	1045	32,33
5	1334	36,52
6	1625	40,31
7	1918	43,79
8	2213	47,04
9	2510	50,10
10	2809	53

Из последнего столбца получим: $(140 + 10)^2 - n = 53^2$, откуда $n = 150^2 - 53^2 = 203 \cdot 97$. Итак, $19\,691 = 203 \cdot 97$, и вычисление потребовало 10 итераций, в каждой из которых было выполнено 1 возведение в степень, 1 вычитание и одно вычисление квадратного корня, т.е. константное число операций.

Естественно, алгоритм Ферма может привести к тому, что по крайней мере один из найденных множителей не будет простым числом. В этом случае необходимо для составного множителя применить снова алгоритм Ферма для разбиения его на множители.

Примечание: Алгоритм Ферма не применим к числам вида $4k+2$ (где k – натуральное число), т.е. числам 6, 10, 14, 18, 22, 26...

Пример 3 иллюстрирует применение алгоритма Ферма для разбиения числа на 2 множителя

Поиск простых чисел с помощью решета Эратосфена

Простое число – это число, делящееся только на единицу и на себя. Определение дает и несложный метод поиска простых путем полного перебора вариантов. Необходимо записать цикл, проходящий все целые числа от 2 до некоторой границы, и для каждого числа пытаться найти нетривиальный (отличный от 1 и

самого числа) делитель. Если такой делитель найден, то число составное, иначе – простое. Более эффективный метод – решето Эратосфена.

Алгоритм:

1. Выписать подряд все целые числа от двух до n (2, 3, 4, ..., n).
2. Пусть переменная $p=2$ – первому простому числу.
3. Зачеркнуть в списке числа, кратные p (это будут числа: $2p, 3p, 4p, \dots$).
4. Найти первое не зачёркнутое число в списке, большее чем p , и присвоить переменной p это значение.
5. Повторять шаги 3 и 4, пока возможно.

Теперь все не зачёркнутые числа в списке – это простые числа от 2 до n .

Пример для $n = 24$.

$a = 2\ 3\ 4\ 5\ 6\ 7\ 8\ 9\ 10\ 11\ 12\ 13\ 14\ 15\ 16\ 17\ 18\ 19\ 20\ 21\ 22\ 23\ 24$

1 Шаг. Находим все числа, кратные 2 и удаляем их:

$a = 2\ 3\ 5\ 7\ 9\ 11\ 13\ 15\ 17\ 19\ 21\ 23$

2 Шаг. Находим все числа, кратные 3 и удаляем их:

$a = 2\ 3\ 5\ 7\ 11\ 13\ 17\ 19\ 23$

3 Шаг. Находим все числа, кратные 5 и удаляем их:

Таких чисел нет, поэтому на этом шаге алгоритм останавливается.

Теперь a состоит только из простых чисел, которые меньше 24.

Пример 4 иллюстрирует работу алгоритма «Решето Эратосфена».

Алгоритмы поиска простых чисел. Числа Мерсенна. Тест Люка-Лемера

Число Мерсённа — число вида $2^n - 1$, где n — натуральное число; такие числа примечательны тем, что некоторые из них являются простыми при больших значениях n . Названы в честь французского математика Марёна Мерсенна, исследовавшего их свойства в XVII веке.

1, 3, 7, 15, 31, 63, 127, 255, 511, 1023, 2047, 4095, 8191, 16 383, 32 767, 65 535, 131 071, ...

Не все числа Мерсенна являются простыми, однако простых чисел среди всех чисел Мерсенна очень много. По этой причине числа Мерсенна удобно использовать для получения большого простого числа.

Для проверки числа Мерсенна на простоту существует простой тест **Люка-Лемера**. Это алгоритм, который предполагает на первом этапе построение последовательности по следующему правилу:

Пусть p — простое нечётное. Число Мерсенна $M_p = 2^p - 1$ простое тогда и только тогда, когда оно делит нацело $(p - 1)$ -й член последовательности

4, 14, 194, 37634, ... [2],

задаваемой рекуррентно:
$$S_k = \begin{cases} 4 & k = 1, \\ S_{k-1}^2 - 2 & k > 1. \end{cases}$$

То есть последовательность всё время одна и та же.

Например, третье число Мерсенна $2^3-1=7$. Для проверки его на простоту с помощью теста Люка-Лемера необходимо построить последовательность (она приведена выше). Третье число Мерсенна (7) должно делить без остатка второй член последовательности (14). Действительно, $14\%7=0$. Математическим языком операция «%» записывается словом mod. То есть можно записать $14(\text{mod } 7)=0$. Поскольку числа в последовательности очень быстро становятся большими, то при программной реализации каждый получаемый член последовательности заменяется на остаток от деления на рассматриваемое число Мерсенна. При этом для получения следующего члена последовательности используется именно остаток от деления с предыдущего шага.

Алгоритм Люка-Лемера (на псевдокоде) выглядит следующим образом:

```

▶Вход: простое нечётное число p
S = 4
k = 1
M = 2p - 1
До тех пока k != p - 1 выполнять
    S = ((S × S) - 2) mod M
    k += 1
Конец цикла
Если S = 0 выполнять
    Возвратить ПРОСТОЕ
иначе
    Возвратить СОСТАВНОЕ
Конец условия

```

Примеры применения теста Люка-Лемера к простому и составному числам Мерсенна:

Число $M_{13} = 2^{13} - 1 = 8191$ — простое^[13]. Действительно,

$$\begin{aligned}
 S_1 &= 4 \\
 S_2 &\equiv 4^2 - 2 \pmod{8191} = 14 \\
 S_3 &\equiv 14^2 - 2 \pmod{8191} = 194 \\
 S_4 &\equiv 194^2 - 2 \pmod{8191} = 4870 \\
 S_5 &\equiv 4870^2 - 2 \pmod{8191} = 3953 \\
 S_6 &\equiv 3953^2 - 2 \pmod{8191} = 5970 \\
 S_7 &\equiv 5970^2 - 2 \pmod{8191} = 1857 \\
 S_8 &\equiv 1857^2 - 2 \pmod{8191} = 36 \\
 S_9 &\equiv 36^2 - 2 \pmod{8191} = 1294 \\
 S_{10} &\equiv 1294^2 - 2 \pmod{8191} = 3470 \\
 S_{11} &\equiv 3470^2 - 2 \pmod{8191} = 128 \\
 S_{12} &\equiv 128^2 - 2 \pmod{8191} = 0
 \end{aligned}$$

Применение теста к числу $M_{11} = 2^{11} - 1 = 2047$

$$\begin{aligned}
 S_1 &= 4 \\
 S_2 &\equiv 4^2 - 2 \pmod{2047} = 14 \\
 S_3 &\equiv 14^2 - 2 \pmod{2047} = 194 \\
 S_4 &\equiv 194^2 - 2 \pmod{2047} = 788 \\
 S_5 &\equiv 788^2 - 2 \pmod{2047} = 701 \\
 S_6 &\equiv 701^2 - 2 \pmod{2047} = 119 \\
 S_7 &\equiv 119^2 - 2 \pmod{2047} = 1877 \\
 S_8 &\equiv 1877^2 - 2 \pmod{2047} = 240 \\
 S_9 &\equiv 240^2 - 2 \pmod{2047} = 282 \\
 S_{10} &\equiv 282^2 - 2 \pmod{2047} = 1736 \neq 0
 \end{aligned}$$

Действительно, $2047 = 23 \cdot 89$.

Пример 5 демонстрирует построение последовательности для теста Люка-Лемера с учетом нахождения остатков от деления на число Мерсенна

Псевдослучайные числа. Числа фон Неймана

В различных технических и математических приложениях часто необходим ряд случайных чисел. Числа называются случайными в силу того, что в их ряду нет никакой закономерности, зная некоторое количество последовательных чисел ряда, нельзя вычислить следующее. Устройство или программа, получающая такие числа, называется генератором случайных чисел. О принципиальной возможности существования случайных последовательностей можно спорить. Нетрудно встретить мнение, что все процессы, существующие в природе или созданные человеком, строго закономерны. Но, во всяком случае, если это и так, то некоторые из закономерностей настолько сложны и управляются настолько большим количеством параметров, что фактически их невозможно отследить за разумное время с использованием разумного количества ресурсов.

Алгоритм фон Неймана для генерации псевдослучайных чисел состоит в следующем. Предположим, что некоторое случайное, достаточно большое число уже известно. Определить такое число совершенно не представляет проблему: так как оно первое, то оно может быть просто любым. Пусть, например, нас интересуют 5-значные случайные числа. Возьмем в качестве первого число 14 563. Возведем его в квадрат, получим 212 080 969. Возьмем из середины пять цифр 20 809. Это и будет следующее случайное число. Псевдослучайность получаемой последовательности очевидна. Каждое следующее число совершенно однозначно определяется предыдущим. Но нас это смущать не должно. Если неизвестно, как получены числа, то они выглядят вполне случайно. Но у метода есть более серьезный недостаток. Если последовательность продолжить, то может проявиться так называемый период. Периодом называется строго повторяющаяся последовательность чисел. Впрочем, для алгоритма фон Неймана можно подобрать исходное число, дающее период только после очень большого количества шагов.

Пример 6 иллюстрирует получение второй, третьей и четвертой цифр пятизначного числа

Пример 1

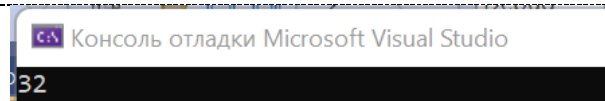
Задача:

Написать на языке C++ программу для поиска наибольшего общего делителя двух чисел методом вычитания

Решение:

```
1  #include <iostream>
2
3  int nod_min(int first_number, int second_number)
4  {
5      int raznost = 0;
6      bool flag = false; //Индикатор окончания цикла
7      while (flag == false)
8      {
9          if (first_number == second_number)
10         {
11             flag = true;
12         }
13         else if (first_number < second_number)
14         {
15             raznost = second_number - first_number;
16             second_number = first_number;
17             first_number = raznost;
18         }
19         else //first_number > second_number
20         {
21             raznost = first_number - second_number;
22             first_number = second_number;
23             second_number = raznost;
24         }
25     }
26     return first_number;
27 }
28
29
30 int main()
31 {
32     std::cout << nod_min(128, 96);
33 }
```

Ответ:



Консоль отладки Microsoft Visual Studio

32

Пример 2

Задача:

Написать на языке C++ программу для нахождения наименьшего положительного целого делителя (большее единицы) для заданного числа A, а также определения того, сколько раз число A будет делиться на этот делитель без остатка (кратность остатка)

Решение:

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      setlocale(LC_ALL, "Russian");
8      unsigned int A; //Заданное число
9      int delitel = 0;
10     int count = 0; //сколько раз найденный делитель
11                     //будет делить без остатка (кратность)
12     cout << "Введите целое положительное число: ";
13     cin >> A;
14     for (int i = 2; i <= sqrt(A); i++)
15     {
16         if (A % i == 0)
17         {
18             delitel = i;
19             while (A % i == 0)
20             {
21                 A = A / i;
22                 count++;
23             }
24             break;
25         }
26     }
27     cout << "Наименьший делитель: " << delitel<<endl;
28     cout << "Кратность делителя: " << count << endl;
29 }

```

Ответ:

Консоль отладки Microsoft Visual Studio

```

Введите целое положительное число: 60
Наименьший делитель: 2
Кратность делителя: 2

```

Пример 3

Задача:

Написать на языке C++ программу для разбиения заданного числа на 2 множителя с помощью алгоритма Ферма

Решение:

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      setlocale(LC_ALL, "Russian");
8      unsigned int n; //Заданное число
9      int delitel1 = 0;
10     int delitel2 = 0;
11     cout << "Введите целое положительное число: ";
12     cin >> n;
13
14     int m = sqrt(n);
15     int x = 1;
16     int q = 0;
17     int koren_iz_q = 0;
18     while(true)
19     {
20         q = (m + x) * (m + x) - n;
21         koren_iz_q = sqrt(q);
22         if (koren_iz_q * koren_iz_q == q)
23         {
24             break;
25         }
26         else
27         {
28             x++;
29         }
30     }
31     delitel1 = (m + x) - koren_iz_q;
32     delitel2 = (m + x) + koren_iz_q;
33
34     cout << n << " = " << delitel1 << " * " << delitel2 << endl;
35 }

```

Ответ:

Консоль отладки Microsoft Visual Studio

```

Введите целое положительное число: 19691
19691 = 97 * 203

```

Пример 4

Задача:

Написать на языке C++ программу для поиска простых чисел, не превосходящих заданную границу, с помощью алгоритма «Решето Эратосфена»

Решение:

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      setlocale(LC_ALL, "Russian");
8      unsigned int n = 0; //Заданное число
9      int delitel1 = 0;
10     int delitel2 = 0;
11     cout << "Введите целое положительное число (границу поиска простых чисел): ";
12     cin >> n;
13
14     int *numbers = new int[n - 1]; //массив для чисел от 2 до n
15     for (int i = 2; i <= n; i++)
16     {
17         numbers[i - 2] = i;
18     }
19
20     int current_index = 0;
21     int current_num = 0;
22
23     while (current_index < n)
24     {
25         if (numbers[current_index] != 0)
26         {
27             current_num = numbers[current_index];
28             for (int i = current_index + 1; i < n; i++)
29             {
30                 if (numbers[i] % current_num == 0)
31                 {
32                     numbers[i] = 0;
33                 }
34             }
35             current_index++;
36         }
37     }
38
39     for (int i = 0; i < n; i++)
40     {
41         if (numbers[i] != 0)
42         {
43             cout << numbers[i] << " ";
44         }
45     }
46     cout << endl;
47
48     delete[] numbers;
49 }

```

Ответ:

Консоль отладки Microsoft Visual Studio

Введите целое положительное число (границу поиска простых чисел): 100
 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Пример 5

Задача:

Написать на языке C++ программу для построения последовательности для теста Люка-Лемера с учетом нахождения остатков от деления на заданное число Мерсенна

Решение:

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int M = 8191; //Число Мерсенна номер 13
8
9      int S = 4; //Первый член последовательности
10     int count = 12; //Количество членов последовательности
11     for (int i = 0; i < count; i++)
12     {
13         cout << S << " ";
14         S = (S * S - 2) % M;
15     }
16     cout << endl;
17 }

```

Ответ:

Консоль отладки Microsoft Visual Studio

```

14 14 194 4870 3953 5970 1857 36 1294 3470 128 0

```

Пример 6

Задача:

Написать на языке C++ программу для получение второй, третьей и четвертой цифр пятизначного числа

Решение:

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main()
6  {
7      int A = 14764;
8      int dig2 = 0;
9      int dig3 = 0;
10     int dig4 = 0;
11
12     int d = A % 10; // Получаем последнюю цифру
13     A /= 10; // Избавляемся от неё
14
15     dig4 = A % 10; // Получаем четвертую цифру
16     A /= 10; // Избавляемся от неё
17
18     dig3 = A % 10; // Получаем третью цифру
19     A /= 10; // Избавляемся от неё
20
21     dig2 = A % 10; // Получаем вторую цифру
22
23     cout << dig2 << dig3 << dig4 << endl;
24 }
25

```

Ответ:

	 Консоль отладки Microsoft Visual Studio 476
Задание 1	
Задача:	
	Написать программу для поиска наибольшего общего делителя (числа вводятся с клавиатуры после запуска программы): Нечетные варианты: наибольший делитель трех чисел методом деления (для поиска остатка от деления в языке C++ используется операция %) Четные варианты: наибольший общий делитель четырех чисел методом вычитания
Решение:	
	<pre data-bbox="279 680 735 1088"> // #1 int max_divisor(int a, int b) { a = abs(a); b = abs(b); while (a % b != 0) { int temp = b; b = a % b; a = temp; } return b; } </pre>
Ответ:	
	2 3 5 => 1 -8 0 16 => 8
Задание 2	
Задача:	
	Написать программу для факторизации заданного с клавиатуры числа методом простого перебора (указать простые множители и их кратность). Для анализа числа на простоту использовать решето Эратосфена
Решение:	


```

// #2
// Функция для нахождения простых чисел методом решета Эратосфена
vector<int> eratosfen(int n)
{
    vector<int> primes;

    // Заполняем массив числами от 2 до n
    for (int i = 2; i ≤ n; i++)
    {
        primes.push_back(i);
    }

    int currentLength = primes.size();
    int previousLength = -1;
    int currIndex = 0;

    // Основной цикл решета Эратосфена
    while (currentLength ≠ previousLength)
    {
        vector<int> newPrimes;

        for (int num : primes)
        {
            if (num % primes[currIndex] ≠ 0 || num = primes[currIndex])
            {
                newPrimes.push_back(num);
            }
        }

        primes = newPrimes;
        previousLength = currentLength;
        currentLength = primes.size();
        currIndex++;
    }

    return primes;
}

// Функция для разложения числа на простые множители
string factor(int n)
{
    vector<int> primes = eratosfen(n);
    reverse(primes.begin(), primes.end());
    string res = "";

    // Цикл по простым числам
    for (int num : primes)
    {
        int divided = 0;

        // Пока число делится на текущий простой множитель
        while (n % num = 0)
        {
            divided++;
            n /= num;
        }

        // Если делитель найден
        if (divided > 0)
        {
            res += to_string(num) + "^" + to_string(divided) + ",";
        }

        // Если не было делений, возвращаем n^1
        res = res.empty() ? to_string(n) + "^1," : res;
    }

    return res.substr(0, res.size() - 1);
}

```

Ответ:

17 = 17¹
 18 = 3²·2¹

Задание 3

Задача:

Написать программу для факторизации заданного с клавиатуры нечетного числа методом Ферма (указать простые множители и их кратность). Для анализа числа на простоту использовать решето Эратосфена

Решение:

```
// #3. Решето Эратосфено взято из II номера
// Вспомогательная функция q_helper
int q_helper(int x, int n, int m)
{
    return (m + x) * (m + x) - n;
}

// Разложение числа на множители методом Ферма
vector<int> factor_by_ferma(int n)
{
    if (n == 1)
    {
        return {};
    }
    vector<int> primes = eratosfen(n);
    if (find(primes.begin(), primes.end(), n) != primes.end())
    {
        return {n};
    }
    if (n % 2 == 0)
    {
        throw invalid_argument("Число не может быть четным (" + to_string(n) + ")");
    }

    int m = static_cast<int>(sqrt(n));
    if (m * m == n)
    {
        return {m, m};
    }

    int x = 1;
    int y;
    while (x < n)
    {
        y = q_helper(x, n, m);
        if (static_cast<int>(sqrt(y)) * static_cast<int>(sqrt(y)) == y)
        {
            break;
        }
        x++;
    }

    int B = static_cast<int>(sqrt(y));
    int A = m + x;
    int p = A + B;
    int q = A - B;

    vector<int> res;
    vector<int> p_factors = (find(primes.begin(), primes.end(), p) != primes.end()) ? vector<int>{p} : factor_by_ferma(p);
    vector<int> q_factors = (find(primes.begin(), primes.end(), q) != primes.end()) ? vector<int>{q} : factor_by_ferma(q);

    res.insert(res.end(), p_factors.begin(), p_factors.end());
    res.insert(res.end(), q_factors.begin(), q_factors.end());

    return res;
}
```

```

// Функция для красивого вывода результата факторизации
✓ string prettify_ferma(const vector<int> &nums)
{
    ... string res = "";
    ... // Уникальные простые множители и их степени
    ✓ ... for (size_t i = 0; i < nums.size(); ++i)
    ... {
    ...     int count = 1;
    ✓ ...     while (i + 1 < nums.size() && nums[i] == nums[i + 1])
    ...     {
    ...         count++;
    ...         i++;
    ...     }
    ...     res += to_string(nums[i]) + "^" + to_string(count) + ",";
    ... }
    ... return res.substr(0, res.size() - 1);
}

```

Ответ:

```

17 = 17^1
135 = 5^1,3^3
Even numbers are not allowed (8)

```

Задание 4

Задача:

Написать программу для проверки на простоту числа Мерсенна с использованием теста Люка-Лемера. С клавиатуры вводится номер числа Мерсенна

Решение:

```

✓ // #4
// Функция для нахождения n, при котором 2^n - 1 = num
✓ int get_mersenn_n(int num)
{
    ... int n = 0;
    ✓ ... while (pow(2, n) - 1 != num)
    ... {
    ...     n++;
    ... }
    ... return n;
}

// Функция Люка-Лемера
✓ int luke_lemer(int k)
{
    ✓ ... if (k == 1)
    ... {
    ...     return 4;
    ... }
    ✓ ... else
    ... {
    ...     return pow(luke_lemer(k - 1), 2) - 2;
    ... }
}

// Проверка числа Мерсенна
✓ bool check_mersenn(int num)
{
    ... int n = get_mersenn_n(num); ... // Находим n для числа Мерсенна
    ... return luke_lemer(n - 1) % num == 0; // Проверяем по критерию Люка-
    ... // Лемера
}

```

Ответ:

15 is Mersenn number: NO
31 is Mersenn number: YES

Задание 5

Задача:

Написать программу для генерации последовательности из 10 пятизначных чисел фон Неймана

Решение:

```
// #5
int neyman(int n)
{
    n *= n;
    string n_str = to_string(n);
    int mid = n_str.length() / 2; // Находим середину строки
    // Извлекаем 5 символов, начиная с mid-2 и заканчивая mid+2
    n = stoi(n_str.substr(mid - 2, 5));
    return n;
}
```

Ответ:

40840 79056 48838 9817 37348 48731 20256 3055 33302 90232

Задание 6*

Задача:

Написать программу для поиска простых чисел по алгоритму «Решето Сундарاما»

Решето Сундарاما — второй по известности алгоритм поиска нечетных простых чисел, придуманный индийским студентом Сундарамом в 1934 году. Его суть состоит в вычеркивании из списка чисел от 1 до $(N - 1)/2$ всех чисел вида

$$i + j + 2ij \\ 1 \leq i \leq j$$

После этого, для оставшихся в списке чисел n , последовательность чисел $2n + 1$ будет представлять все нечетные простые числа до N .

вычеркиваем $1+j+2*1*j$: [1, 2, 3, ~~4~~, 5, 6, ~~7~~, 8, 9, ~~10~~, ...]

вычеркиваем $2+j+2*2*j$: [1, 2, 3, 5, 6, 8, 9, 11, ~~12~~, 14, 15, ...]

вычеркиваем $3+j+2*3*j$: [1, 2, 3, 5, 6, 8, 9, 10, 11, 14, ..., ~~24~~, ...]

и так далее...

последовательность $2n+1$: [3, 5, 7, 11, 13, 17, 19, ...]

Решение:

```

// #6
vector<int> sundaram(int n)
{
    ....int upper = floor((n - 1) / 2); // Верхняя граница
    ....vector<int> primes(upper);.....// Массив для простых чисел
    ....vector<int> indexes;.....// Массив для хранения индексов, которые
        будут исключены

    ....// Инициализация массива простых чисел
    ✓ ....for (int i = 1; i ≤ upper; ++i)
    ....{
    ....    primes[i - 1] = i;
    ....}

    ....// Генерация индексов, которые будут исключены
    ✓ ....for (int i = 1; i < n; ++i)
    ....{
    ✓ ....    for (int j = i; j < n; ++j)
    ....        {
    ....            ....int k = i + j + 2 * i * j;
    ✓ ....            ....if (k ≤ upper)
    ....                {
    ....                    ....indexes.push_back(k);
    ....                }
    ✓ ....            ....else
    ....                {
    ....                    ....break;
    ....                }
    ....        }
    ....}

    ....// Помечаем исключённые индексы как -1
    ✓ ....for (int i = 0; i < indexes.size(); ++i)
    ....{
    ✓ ....    ....if (indexes[i] ≤ upper)
    ....        {
    ....            ....primes[indexes[i] - 1] = -1;
    ....        }
    ....}

    ....// Формирование итогового списка простых чисел
    ....vector<int> result = {2}; // 2 является простым числом
    ✓ ....for (int i = 0; i < primes.size(); ++i)
    ....{
    ✓ ....    ....if (primes[i] ≠ -1)
    ....        {
    ....            ....result.push_back(2 * primes[i] + 1); // Преобразуем по формуле
                2n + 1
    ....        }
    ....}

    ....return result;
}

```

Ответ:

25 => 2 3 5 7 11 13 17 19 23